Mel-Frequency Cepstral Coefficients (MFCCs) for Speech Recognition

Lab 11

Department of Artificial Intelligence
NUML — Islamabad

Submitted by: Junaid Asif

Roll No: BSAI–144

Semester: 7th

Instructor: Dr. Sajjad Ghauri

December 11, 2025

Objective

To understand and implement the complete process of extracting Mel-Frequency Cepstral Coefficients (MFCCs) from a speech signal using MATLAB and to analyze their application in speech recognition and speaker identification.

Introduction

Mel-Frequency Cepstral Coefficients (MFCCs) are among the most widely used features in speech and speaker recognition systems. MFCCs model the human auditory perception by emphasizing lower frequencies more than higher ones. The MFCC extraction process transforms raw speech signals into a compact and discriminative feature representation.

MFCC Extraction Procedure

Step 1: Framing

```
% Load speech signal
[speechSignal, fs] = audioread('speech.wav');

% Frame parameters
frameDuration = 0.025;   % 25 ms
frameOverlap  = 0.010;   % 10 ms

frameLen = round(frameDuration * fs);
frameStep = round(frameOverlap * fs);

numFrames = floor((length(speechSignal)-frameLen)/frameStep) + 1;
frames = zeros(frameLen, numFrames);

for i = 1:numFrames
    startIndex = (i-1)*frameStep + 1;
    endIndex = startIndex + frameLen - 1;
    frames(:,i) = speechSignal(startIndex:endIndex);
end
```

Explanation: Framing divides the non-stationary speech signal into short-time stationary segments.

Step 2: Windowing

```
window = hamming(frameLen);
windowedFrames = frames .* repmat(window,1,numFrames);
```

Explanation: Windowing reduces spectral leakage caused by abrupt frame boundaries.

Step 3: FFT

```
NFFT = 512;
magSpectrum = abs(fft(windowedFrames, NFFT));
```

Explanation: FFT converts each frame from time domain to frequency domain.

Step 4: Mel Filter Bank

```
numFilters = 26;
lowFreq = 0;
highFreq = fs/2;

melLow  = 2595*log10(1 + lowFreq/700);
melHigh = 2595*log10(1 + highFreq/700);

melPoints = linspace(melLow, melHigh, numFilters+2);
hzPoints = 700*(10.^(melPoints/2595)-1);

filterBank = zeros(numFilters, NFFT/2+1);

for i = 1:numFilters
    for j = 1:(NFFT/2+1)
        freq = (j-1)*fs/NFFT;
        if freq >= hzPoints(i) && freq <= hzPoints(i+1)
            filterBank(i,j) = (freq-hzPoints(i))/(hzPoints(i+1)-
                hzPoints(i));
        elseif freq > hzPoints(i+1) && freq <= hzPoints(i+2)
            filterBank(i,j) = (hzPoints(i+2)-freq)/(hzPoints(i+2)-
                hzPoints(i+1));
        end
    end
end

filterBankEnergy = filterBank * magSpectrum(1:NFFT/2+1,:);
```

Step 5: Logarithm of Filter Bank Energies

```
logEnergy = log(filterBankEnergy + eps);
```

Step 6: DCT

```
numCepCoeff = 13;
mfccs = dct(logEnergy);
mfccs = mfccs(1:numCepCoeff,:);
```

Task 1: Visualizing Speech Features

```
figure;
imagesc(mfccs);
xlabel('Frame Index');
ylabel('Cepstral Coefficient');
title('MFCC Features');
colorbar;
```

Observation: Different speech sounds produce distinct MFCC patterns.

Task 2: Effect of Frame Size and Overlap

```
frameDurations = [0.015 0.025 0.04];
overlaps = [0 0.01 0.02];

for fd = frameDurations
    for ov = overlaps
        frameLen = round(fd * fs);
        frameStep = round(ov * fs);
        % Repeat MFCC extraction here
    end
end
```

Discussion: Smaller frames improve temporal resolution while larger frames improve spectral detail.

Task 3: Effect of Number of Mel Filters and Cepstral Coefficients

```
melFilters = [20 26 40];
cepCoeff   = [10 13 20];

for nf = melFilters
    for nc = cepCoeff
        numFilters = nf;
        numCepCoeff = nc;
        % Recompute MFCCs
    end
end
```

Discussion: Increasing Mel filters improves frequency resolution while more cepstral coefficients capture finer spectral details.

Task 4: Basic Speaker Identification Using MFCCs

```
% Compute average MFCC vector for each speaker
speaker1_mean = mean(mfccs_speaker1, 2);
speaker2_mean = mean(mfccs_speaker2, 2);

% Test sample
test_mean = mean(mfccs_test, 2);

% Euclidean distance
d1 = norm(test_mean - speaker1_mean);
d2 = norm(test_mean - speaker2_mean);

if d1 < d2
    disp('Speaker 1 Identified');
else
    disp('Speaker 2 Identified');
end
```

Explanation: The speaker whose MFCC model has minimum distance is identified as the speaker.

Conclusion

This lab implemented the complete MFCC extraction pipeline and demonstrated its effectiveness in speech analysis and basic speaker identification. MFCCs provide a compact and perceptually meaningful representation of speech signals, making them highly suitable for speech recognition applications.